SOFTWARE TESTING

https://chatgpt.com/share/69947c00-ab94-8011-9b89-e886d17f88d0

✅ PART-1 (FIRST 5 QUESTIONS)

🔷 Q1

🔹 QUESTION

(Chapter-04: Control Flow Graph & Cyclomatic Complexity)

Draw the Control Flow Graph (CFG) for the following program and compute Cyclomatic Complexity. Also list independent paths.

```
Start
if (A > 10)
  B = B + 1
else
  B = B - 1
if (B > 5)
  print("Hi")
Stop
```

🔹 STEP-1: THEORY EXPLANATION

✅ Intuition

We count how many logical routes exist in a program.

More branches → more paths → more test cases.

✅ Basic Explanation

CFG shows flow of program using nodes and arrows.

Each decision creates new paths.

✅ Exam Definition

Cyclomatic Complexity measures number of linearly independent paths.

✅ Real-Life Analogy

Think of Google Maps routes from home to college.

More turns → more possible routes.

✅ Practical Use

Helps decide minimum test cases.

✅ Important Exam Points

• Draw CFG neatly
• Count decisions
• Use formula correctly

🔹 STEP-2: DIAGRAM (CFG)

```
(1 Start)
   |
[A>10?]----No---->(5)
   |Yes             |
  (3)              (6)
   |                |
   ------> (6 B>5?) ----No----> (8 Stop)
                |
               Yes
                |
             (7 print)
                |
             (8 Stop)
```

✔ Labels Explained

[A>10?] = Decision 1
[B>5?] = Decision 2
Nodes = statements
Arrows = control flow

🔹 STEP-3: FORMULAS

V(G) = Decisions + 1

Where:
V(G) = Cyclomatic Complexity

🔹 STEP-4: CALCULATION

Decisions:

1. A > 10
2. B > 5

Total = 2

So,

V(G) = 2 + 1
V(G) = 3

Independent Paths

Path-1: A>10 → B>5
Path-2: A>10 → B≤5
Path-3: A≤10 → B>5

🔹 STEP-5: FINAL ANSWER

👉 FINAL ANSWER:
Cyclomatic Complexity = **3**
Minimum test cases = **3**

🔹 STEP-6: EXAM TIP

✓ Draw CFG clearly
✓ Circle decision nodes
✓ Write formula
✓ Students forget +1

🔷 Q2

🔹 QUESTION

(Chapter-01: Decision Table Testing)

Design a decision table for Library Book Issue:

Conditions:
• Member valid
• Book available
• Fine cleared

Action: Issue book only if all true.

◆ STEP-1 THEORY

Intuition

We test combinations of conditions.

Definition

Decision table maps conditions to actions.

Real-Life

Library clerk checks all three before issuing.

Use

Avoids missing test cases.

Exam Points

• Conditions top
• Actions bottom

◆ STEP-2: DECISION TABLE

```
Conditions:

           R1  R2  R3  R4
Member      T   T   F   T
Book        T   F   T   T
Fine        T   T   T   F

Action:
Issue       Y   N   N   N
```

Labels

T = True
F = False
Y = Issue book
N = Do not issue

🔹 STEP-3 FORMULA

Total combos = $2^3 = 8$
Reduced to 4 important rules.

🔹 STEP-4 CALCULATION

Only R1 satisfies all.

🔹 STEP-5 FINAL

👉 FINAL ANSWER:
4 rules sufficient.

🔹 STEP-6 TIP

✓ Draw table lines properly
✓ Mention reduced rules
✓ Don't write all 8 unless asked

🔷 Q3

🔹 QUESTION

(Chapter-02: Cause–Effect Graph)

A student passes exam if:
• Attendance ≥75% (C1)
• Internal marks ≥40 (C2)
• External marks ≥40 (C3)

Construct cause-effect graph and derive test cases.

🔹 STEP-1 THEORY

Cause = Input condition
Effect = Output

Graph connects them.

Real-Life

All conditions needed to pass.

Exam Points

AND gate important.

◆ STEP-2 DIAGRAM

```
C1 ----\
        AND ----> PASS
C2 ----/
        \
         AND
        /
C3 ----/

FAIL = NOT PASS
```

Labels

C1 Attendance
C2 Internal
C3 External

◆ STEP-3 FORMULA

Total combos = $2^3 = 8$
Minimal tests = 4

◆ STEP-4 TEST CASES

C1	C2	C3	Result
T	T	T	Pass
F	T	T	Fail

C1	C2	C3	Result
T	F	T	Fail
T	T	F	Fail

🔹 STEP-5 FINAL

👉 FINAL ANSWER: 4 test cases.

🔹 STEP-6 TIP

✓ Draw AND gate
✓ Write fail logic

🔷 Q4

🔹 QUESTION

(Chapter-04: MC/DC Testing)

For condition:

```
(A OR B)
```

Find minimum MC/DC test cases.

🔹 STEP-1 THEORY

Each variable must independently affect output.

🔹 STEP-2 TABLE

```
A B | Output
T F | T
F T | T
F F | F
```

🔹 STEP-3 FORMULA

MC/DC tests = N + 1
N = conditions

N=2
Tests=3

🔹 STEP-4 CALCULATION

Change A → output changes
Change B → output changes

🔹 STEP-5 FINAL

👉 FINAL ANSWER: 3 tests.

🔹 STEP-6 TIP

✓ Show independence
✓ Students wrongly write 4

🔷 Q5

🔹 QUESTION

(Chapter-08: Mutation Testing)

Total mutants = 60
Killed mutants = 45

Find mutation score.

🔹 STEP-1 THEORY

Mutation score shows test strength.

🔹 STEP-2 DIAGRAM

```
Original Program
   |
Generate Mutants (60)
   |
Killed = 45
Alive = 15
```

STEP-3 FORMULA

Mutation Score (MS)

MS = (Killed / Total) × 100

STEP-4 CALCULATION

MS = (45 / 60) × 100
MS = 0.75 × 100
MS = 75%

STEP-5 FINAL

👉 FINAL ANSWER: 75%

Q6

QUESTION

(Chapter-05: Concolic Testing)

Explain Concolic Testing. For the program below, show how paths are generated.

```
if (x > 5)
   if (y > 10)
       print("A")
   else
       print("B")
```

```
else
  print("C")
```

Find number of paths and test inputs.

◆ STEP-1: THEORY EXPLANATION

✅ Intuition

Concolic = Concrete + Symbolic testing.

It runs program with real inputs and also tracks symbolic conditions.

✅ Basic Explanation

• Concrete = actual numbers
• Symbolic = variables like x_0, y_0
• Solver finds new inputs

✅ Exam Definition

Concolic testing combines concrete execution and symbolic execution to explore paths.

✅ Real-Life Analogy

Like solving a maze by walking AND marking directions.

✅ Practical Use

Automatic test case generation.

✅ Important Exam Points

• Mention constraint solver
• Mention path constraints
• Mention iteration

◆ STEP-2: DIAGRAM (Path Tree)

```
          Start
            |
         x>5?
        /    \
      T        F
     /          \
```

```
  y>10?        C
  /   \
T      F
|      |
A      B
```

✓ Labels

x>5 → Condition 1
y>10 → Condition 2

🔹 STEP-3: FORMULA

Paths = number of leaf nodes

🔹 STEP-4: CALCULATION

Path-1: x>5, y>10 → A
Test: x=6, y=11

Path-2: x>5, y≤10 → B
Test: x=6, y=5

Path-3: x≤5 → C
Test: x=3

Total paths = 3

🔹 STEP-5: FINAL ANSWER

👉 FINAL ANSWER:
3 paths, 3 test cases.

🔹 STEP-6: EXAM TIP

✓ Draw path tree
✓ Write constraints
✓ Mention solver

🔷 Q7

🔹 QUESTION

(Chapter-03: Orthogonal Array Testing)

A system has 3 inputs:
Browser (Chrome, Firefox)
OS (Windows, Linux)
Network (WiFi, LAN)

Design OATS test cases.

◆ STEP-1 THEORY

Intuition

Test all pairs efficiently.

Definition

Orthogonal Array Testing ensures pairwise coverage.

Real-Life

Testing phone on different SIM + network combos.

Practical Use

Reduces testing cost.

Exam Points

• Mention pairwise
• Mention factors & levels

◆ STEP-2: TABLE

Factors = 3
Levels = 2

```
Test Browser OS Network
T1     C       W   WiFi
T2     C       L   LAN
T3     F       W   LAN
T4     F       L   WiFi
```

Labels

C=Chrome
F=Firefox
W=Windows
L=Linux

🔹 STEP-3 FORMULA

Without OATS:
2^3 = 8 tests

With OATS:
Only 4 tests

🔹 STEP-4 CALCULATION

All pairs covered:

Browser-OS ✓
Browser-Network ✓
OS-Network ✓

🔹 STEP-5 FINAL

👉 FINAL ANSWER:
4 tests sufficient.

🔹 STEP-6 TIP

✓ Write pairwise word
✓ Show reduction

🔷 Q8

🔹 QUESTION

(Chapter-06: Data Flow Testing)

For the code:

```
1. a=5
2. b=a+2
3. if(b>6)
```

```
4.   c=a+b
5. print(c)
```

Find DU chains.

🔹 STEP-1 THEORY

Intuition

Track variable life.

Definition

DU chain = Definition → Use without redefinition.

Real-Life

Like using money after earning.

Practical

Find data misuse bugs.

Exam Points

• Identify DEF
• Identify USE

🔹 STEP-2 DIAGRAM (Data Flow)

```
a defined → used in 2 → used in 4
b defined → used in 3 & 4
c defined → used in 5
```

🔹 STEP-3 FORMULA

DU = (DEF, USE)

🔹 STEP-4 CALCULATION

a: (1→2), (1→4)
b: (2→3), (2→4)
c: (4→5)

Total DU chains = 5

🔹 STEP-5 FINAL

👉 FINAL ANSWER:
5 DU chains.

🔹 STEP-6 TIP

✓ Write DEF & USE separately
✓ Students miss indirect uses

🔷 Q9

🔹 QUESTION

(Chapter-12: Integration Testing)

Explain Top-Down Integration with diagram.

🔹 STEP-1 THEORY

Intuition

Test from top module first.

Definition

Integrate modules from main control downward.

Real-Life

Manager tested before workers.

Practical

Early interface detection.

Exam Points

• Mention stubs
• Mention regression testing

🔹 STEP-2 DIAGRAM

```
        A
      /   \
    B       C
   / \
  D   E
```

Top-Down Order:

A → B → D → E → C

Stubs replace lower modules first.

Labels

A = Main module
B,C = submodules
D,E = leaf modules

🔹 STEP-3 FORMULA

No formula.

🔹 STEP-4 CALCULATION

Testing sequence:

1. Test A with stubs
2. Replace B
3. Replace D & E
4. Replace C

🔹 STEP-5 FINAL

👉 FINAL ANSWER:
Top-Down starts from main using stubs.

🔹 STEP-6 TIP

✓ Draw tree
✓ Write order
✓ Mention stubs

Q10

QUESTION

(Chapter-04: Basis Path Testing)

Find independent paths.

```
if(A)
   X
else
   Y
print(Z)
```

STEP-1 THEORY

Intuition

Cover all logical routes.

Definition

Basis path ensures all independent paths tested.

Real-Life

Taking different roads to same place.

Practical

Ensures high coverage.

Exam Points

• Count decisions
• V(G)=D+1

STEP-2 CFG

```
Start
  |
 A?
/   \
```

```
X     Y
 \   /
  Z
```

◆ STEP-3 FORMULA

V(G)=Decisions+1

◆ STEP-4 CALCULATION

Decisions=1

V(G)=1+1=2

Paths:

P1: A→X→Z
P2: A→Y→Z

◆ STEP-5 FINAL

👉 FINAL ANSWER:
2 independent paths.

◆ Q11

◆ QUESTION

(Chapter-11: FSM Based Testing / Unit Testing Level)

A simple vending machine works as follows:

• Coin inserted → State S1
• Select item → State S2
• Item delivered → State S0 (start)

Draw FSM and design test cases.

🔹 STEP-1: THEORY EXPLANATION

✅ Intuition

FSM = system moves between states.

✅ Basic Explanation

State = condition of system
Transition = movement between states

✅ Exam Definition

Finite State Machine (FSM) models system behavior using states and transitions.

✅ Real-Life Analogy

ATM moves: Idle → Card → PIN → Cash.

✅ Practical Use

Used in UI, protocols, machines.

✅ Important Exam Points

• Show states
• Show transitions
• Label arrows

🔹 STEP-2: FSM DIAGRAM

```
      Coin
  S0 ------> S1
              |
           Select
              v
```

```
          S2
           |
      Deliver Item
           v
          S0
```

✓ Labels Explained

S0 = Idle
S1 = Coin inserted
S2 = Selection done

🔹 STEP-3: FORMULA

No formula.

🔹 STEP-4: TEST CASES

Test	Input	Expected State
T1	Coin	S1
T2	Coin+Select	S2
T3	Coin+Select+Deliver	S0

🔹 STEP-5: FINAL ANSWER

👉 FINAL ANSWER:
3 test cases cover FSM.

🔹 STEP-6: EXAM TIP

✓ Draw circles for states
✓ Arrows for transitions
✓ Label inputs on arrows

🔷 Q12

🔹 QUESTION

(Chapter-01: Decision Table Testing – Washing Machine)

Washing machine rules:

• Power ON
• Water available
• Door closed

Machine runs only if all YES.

Construct decision table.

◆ STEP-1 THEORY

Decision table tests condition combinations.

Real-Life

Machine won't run if door open.

Exam Points

• Conditions top
• Actions bottom

◆ STEP-2: DECISION TABLE

```
Conditions:

        R1  R2  R3  R4
Power    T   T   F   T
Water    T   F   T   T
Door     T   T   T   F

Action:
Run      Y   N   N   N
```

Labels

T=True
F=False
Y=Run
N=Stop

◆ STEP-3 FORMULA

$2^3 = 8$ combos
Reduced to 4 rules.

🔹 STEP-4 CALCULATION

Only R1 satisfies all.

🔹 STEP-5 FINAL

👉 FINAL ANSWER:
4 rules sufficient.

🔹 STEP-6 TIP

✓ Draw proper grid
✓ Mention reduced rules

🔷 Q13

🔹 QUESTION

(Chapter-08: Mutation Testing – Table Based)

Original statement:

```
if(a>b)
```

Mutants created:

1. a>=b
2. a<b
3. a==b
4. a<=b

Testing kills 3 mutants.
Find mutation score.

🔹 STEP-1 THEORY

Mutation testing checks test power.

Real-Life

Like checking exam traps.

Practical

Stronger test suite.

🔹 STEP-2: MUTATION TABLE

```
Original → if(a>b)

Mutants:
M1 a>=b
M2 a<b
M3 a==b
M4 a<=b
```

Killed = 3
Alive = 1

🔹 STEP-3 FORMULA

MS = (Killed / Total) ×100

🔹 STEP-4 CALCULATION

MS = (3/4) ×100
MS = 75%

🔹 STEP-5 FINAL

👉 FINAL ANSWER: 75%

🔹 STEP-6 TIP

✓ Write formula
✓ Show division
✓ Write % sign

🔷 Q14

◆ QUESTION

(Chapter-14: System Testing – Alpha vs Beta)

Explain Alpha and Beta testing with diagram.

◆ STEP-1 THEORY

Alpha

Testing inside company.

Beta

Testing by real users.

Real-Life

Game tested by players before release.

Practical

Find real-world bugs.

Exam Points

• Mention environment
• Mention users

◆ STEP-2 DIAGRAM

```
Developer → Alpha → Beta → Release
   |          |        |
Internal   Testers   Users
```

Labels

Alpha = controlled
Beta = real environment

◆ STEP-3 FORMULA

No formula.

🔹 STEP-4 CALCULATION

Not numerical.

🔹 STEP-5 FINAL

👉 FINAL ANSWER:
Alpha = internal
Beta = external users

🔹 STEP-6 TIP

✓ Draw flow
✓ Write differences

🔷 Q15

🔹 QUESTION

(Chapter-12: Bottom-Up Integration Testing)

Explain Bottom-Up integration with diagram.

🔹 STEP-1 THEORY

Intuition

Test lower modules first.

Definition

Integration starts from leaf modules.

Real-Life

Build house from foundation.

Practical

No stubs needed.

Exam Points

• Mention drivers
• Mention sequence

◆ STEP-2 DIAGRAM

```
       A
     /   \
    B     C
   / \
  D   E
```

Bottom-Up Order:

D,E → B → C → A

Drivers simulate higher modules.

Labels

A=Top
D,E=Leaf

◆ STEP-3 FORMULA

No formula.

◆ STEP-4 CALCULATION

Sequence testing:

1. D,E
2. B
3. C
4. A

◆ STEP-5 FINAL

👉 FINAL ANSWER:
Bottom-Up starts from leaf using drivers.

G

🔷 Q16

🔹 QUESTION

(Chapter-13: Call Graph Based Integration Testing)

Consider modules A, B, C, D where:

- A calls B and C
- B calls D
- C calls D

Draw Call Graph and find number of pair-wise integration test sessions.

🔹 STEP-1: THEORY EXPLANATION

✅ Intuition

Call graph shows which module calls which.

✅ Basic Explanation

Node = module
Arrow = call relation

✅ Exam Definition

Call graph is a directed graph representing calling relationships among modules.

✅ Real-Life Analogy

Manager calls supervisors, supervisors call workers.

✅ Practical Use

Avoids unnecessary stubs/drivers.

✅ Important Exam Points

• Nodes = modules

• Edges = calls

• Pair-wise = each call tested

🔹 STEP-2: DIAGRAM

```
    A
   / \
  B   C
   \ /
    D
```

✓ Labels Explained

A = main module

B,C = intermediate

D = common module

🔹 STEP-3: FORMULA

Pair-wise sessions = Number of edges

🔹 STEP-4: CALCULATION

Edges:

A→B (1)
A→C (2)
B→D (3)
C→D (4)

Total = 4

🔹 STEP-5: FINAL ANSWER

👉 FINAL ANSWER:
4 pair-wise sessions.

◆ STEP-6: EXAM TIP

✓ Count arrows carefully
✓ Students miss shared calls

◆ Q17

◆ QUESTION

(Chapter-13: Path-Based Integration Testing)

Explain MM-Path and identify MM-paths for:

Module A calls B, then C.

◆ STEP-1: THEORY

Intuition

MM-path tracks execution across modules.

Definition

MM-path = Module Execution Path + Messages (calls).

Real-Life

Passing a file from one office to another.

Practical

Finds inter-module bugs.

Exam Points

• Mention message
• Mention control transfer

◆ STEP-2: DIAGRAM

```
A ---> B ---> C
```

Labels

Arrows = messages
Boxes = modules

🔹 STEP-3: FORMULA

MM-path = sequence of modules crossed.

🔹 STEP-4: CALCULATION

Path-1: A→B→C
Path-2: A→C (if direct call allowed)

Total MM paths = 2

🔹 STEP-5: FINAL ANSWER

👉 FINAL ANSWER:
2 MM-paths.

🔹 STEP-6: EXAM TIP

✓ Draw module boxes
✓ Label messages

🔷 Q18

🔹 QUESTION

(Chapter-07: Data Flow Anomalies)

Explain anomalies for sequence:

```
d u d k
```

🔹 STEP-1 THEORY

Intuition

Track variable life.

Definition

d = define
u = use
k = kill

Real-Life

Write → use → rewrite → delete.

Practical

Find logical bugs.

Exam Points

• Define before use
• Avoid unused definitions

🔹 STEP-2: DIAGRAM

```
d → u → d → k
```

Labels

d=definition
u=use
k=kill

🔹 STEP-3 FORMULA

Valid sequence rule:

d must come before u.

🔹 STEP-4 CALCULATION

Sequence:

d→u ✓ valid
u→d ✓ allowed
d→k ✓ valid

No anomaly.

🔹 STEP-5 FINAL

👉 FINAL ANSWER:
No anomaly.

🔹 STEP-6 TIP

✓ Write symbol meanings
✓ Students confuse k

🔷 Q19

🔹 QUESTION

(Chapter-04: MC/DC Advanced)

For condition:

```
(A AND B) OR C
```

Find minimum MC/DC tests.

🔹 STEP-1 THEORY

Each condition must independently change output.

Real-Life

Each switch should control light.

Exam Points

- Show independence
- Use truth table

🔹 STEP-2 TABLE

```
A B C | Output
T T F | T
T F F | F
F T F | F
T T T | T
```

◆ STEP-3 FORMULA

MC/DC tests = N + 1
N=3
Tests=4

◆ STEP-4 CALCULATION

Change A → affects output
Change B → affects output
Change C → affects output

◆ STEP-5 FINAL

👉 FINAL ANSWER:
4 test cases.

◆ STEP-6 TIP

✓ Show each condition effect
✓ Don't skip truth table

◆ Q20

◆ QUESTION

(Chapter-05: Concolic Testing Advanced)

For:

```
if(x>0)
 if(y>0)
   Z=1
 else
   Z=2
```

Find path constraints and test inputs.

◆ STEP-1 THEORY

Concolic combines real and symbolic runs.

Practical

Auto-generate tests.

Exam Points

• Mention constraints
• Mention solver

◆ STEP-2 DIAGRAM

```
      x>0?
     /   \
   T       F
  /         \
y>0?         End
/   \
1    2
```

◆ STEP-3 FORMULA

Paths = leaves

◆ STEP-4 CALCULATION

Path-1: x>0, y>0
Test: (1,1)

Path-2: x>0, y≤0
Test: (1,-1)

Path-3: x≤0
Test: (-1,0)

Total = 3

◆ STEP-5 FINAL

👉 FINAL ANSWER:
3 paths, 3 tests.

🔷 Q21

🔹 QUESTION

(Chapter-12: Integration Testing Effort Calculation)

A module structure has:

Nodes = 10
Leaves = 4
Edges = 9

Find the number of integration test sessions.

🔹 STEP-1: THEORY EXPLANATION

✅ Intuition

Integration effort counts how many times we must test module combinations.

✅ Basic Explanation

Each session = one integration configuration.

✅ Exam Definition

Integration effort = number of test sessions required in decomposition-based integration.

✅ Real-Life Analogy

Like assembling machine parts step-by-step and testing each time.

✅ Practical Use

Helps project planning.

✅ Important Exam Points

• Use correct formula
• Identify nodes/leaves/edges correctly

🔹 STEP-2: DIAGRAM (Example Structure)

```
        A
      /   \
     B      C
    / \      \
   D   E      F
```

(Just to visualize hierarchy)

✓ Labels

Nodes = modules
Leaves = lowest modules

🔹 STEP-3: FORMULA

Integration Effort:

```
Sessions = Nodes − Leaves + Edges
```

Where:

Nodes = total modules
Leaves = modules calling nobody
Edges = connections

🔹 STEP-4: CALCULATION

Given:

Nodes = 10
Leaves = 4
Edges = 9

So:

Sessions = 10 − 4 + 9
Sessions = 6 + 9
Sessions = 15

🔹 STEP-5: FINAL ANSWER

👉 FINAL ANSWER: **15 test sessions**

🔹 STEP-6: EXAM TIP

✓ Write formula first
✓ Substitute values clearly
✓ Students forget leaves subtraction

🔷 Q22

🔹 QUESTION

(Chapter-08: Mutation Testing – Equivalent Mutants)

Total mutants = 20
Equivalent mutants = 5
Killed mutants = 10

Find Mutation Score.

🔹 STEP-1: THEORY

Intuition

Equivalent mutants cannot be killed.

Definition

Mutation score measures test effectiveness.

Real-Life

Trick questions in exam that look different but same answer.

Practical

Improves test quality.

Exam Points

Exclude equivalent mutants.

🔹 STEP-2: DIAGRAM

```
Total Mutants = 20
Equivalent = 5 (ignored)
Testable = 15
Killed = 10
```

🔹 STEP-3: FORMULA

```
MS = (Killed / (Total − Equivalent)) × 100
```

MS = Mutation Score

🔹 STEP-4: CALCULATION

Testable mutants:

20 − 5 = 15

Now:

MS = (10 / 15) × 100
MS = 0.666 × 100
MS = 66.6%

🔹 STEP-5: FINAL ANSWER

👉 FINAL ANSWER: **66.6%**

🔹 STEP-6: EXAM TIP

✓ Subtract equivalent mutants
✓ Show decimal
✓ Write % sign

🔷 Q23

◆ QUESTION

(Chapter-14: System Testing – Smoke Testing)

Explain Smoke Testing with diagram and example.

◆ STEP-1: THEORY

Intuition

Quick check before deep testing.

Definition

Smoke testing verifies basic functionality before detailed testing.

Real-Life

Turning on car before driving.

Practical

Prevents wasting time on broken builds.

Exam Points

• Early testing
• Basic features only

◆ STEP-2: DIAGRAM

```
Build Ready
    |
Smoke Test
    |
Pass → Full Testing
Fail → Fix Build
```

✓ Labels

Smoke Test = basic checks

◆ STEP-3: FORMULA

No formula.

🔹 STEP-4: CALCULATION

Example checks:

✓ Login works
✓ Button clickable
✓ Database connects

🔹 STEP-5: FINAL ANSWER

👉 FINAL ANSWER:
Smoke testing = quick basic verification.

🔹 STEP-6: EXAM TIP

✓ Draw flow diagram
✓ Mention early stage
✓ Students confuse with regression

🔷 Q24

🔹 QUESTION

(Chapter-03: Orthogonal Array Testing – Advanced)

Three inputs:

Browser (C,F,S)
OS (W,L,M)
Network (WiFi,Lan,5G)

Design OATS.

🔹 STEP-1 THEORY

Intuition

Test pairs, not all combos.

Definition

OATS ensures pairwise coverage.

Real-Life

Phone tested on network+OS combos.

Practical

Reduces cost.

Exam Points

• Factors
• Levels

◆ STEP-2: OATS TABLE

```
Test Browser OS Network
T1     C       W   WiFi
T2     C       L   Lan
T3     C       M   5G
T4     F       W   Lan
T5     F       L   5G
T6     F       M   WiFi
T7     S       W   5G
T8     S       L   WiFi
T9     S       M   Lan
```

✓ Labels

C=Chrome
F=Firefox
S=Safari
W=Windows
L=Linux
M=Mac

◆ STEP-3 FORMULA

Without OATS:

$3^3 = 27$ tests

With OATS:

Only 9

🔹 STEP-4 CALCULATION

All pairs covered.

🔹 STEP-5 FINAL

👉 FINAL ANSWER: **9 tests sufficient**

🔹 STEP-6 TIP

✓ Show reduction
✓ Write pairwise term

🔷 Q25

🔹 QUESTION

(Chapter-04: Basis Path Testing – Full CFG)

For:

```
if(A)
   if(B)
      X
   else
      Y
Z
```

Draw CFG and find independent paths.

🔹 STEP-1 THEORY

Intuition

Test all logical routes.

Definition

Basis path testing ensures path coverage.

Real-Life

Different roads to same destination.

Practical

Improves coverage.

Exam Points

• Draw CFG
• Count decisions

🔹 STEP-2 CFG

```
   Start
     |
    A?
   /  \
  T    F
  |    |
  B?   |
 / \   |
X   Y  |
 \   \ /
    Z
```

✔ Labels

A?, B? = decisions

🔹 STEP-3 FORMULA

```
V(G) = Decisions + 1
```

🔹 STEP-4 CALCULATION

Decisions:

A = 1
B = 1

Total = 2

So:

V(G) = 2 + 1 = 3

Independent Paths

P1: A→B→X→Z
P2: A→B→Y→Z
P3: A→Z

🔹 STEP-5 FINAL

👉 FINAL ANSWER:
3 independent paths.

🔹 STEP-6 TIP

✔ Draw neat CFG
✔ List paths clearly
✔ Students miss A→Z path

If this is good, say:

"Continue Part-6"

Final part will include:

✔ Cause-effect advanced
✔ Data-flow deep
✔ MC/DC full table
✔ Concolic solver style
✔ Integration strategy comparison PYQs

🔷 Q26

🔹 QUESTION

(Chapter-02: Cause–Effect Graph Testing)

An online exam system follows these rules:

• Student must be **registered**
• Exam fee must be **paid**
• Exam window must be **open**

Only if all conditions are satisfied, the student can **start the exam**.

Construct:

1. Cause–Effect Graph
2. Decision Table
3. Test Cases

🔹 STEP-1: THEORY EXPLANATION (FROM ZERO LEVEL)

✅ Intuition

We check **conditions (causes)** to decide an **output (effect)**.

✅ Basic Explanation

Cause–Effect Graph connects input conditions logically to outputs.

✅ Exam-level Definition

Cause–Effect Graphing is a black-box testing technique that models logical relationships between causes and effects using Boolean logic.

✅ Real-Life Analogy

You can enter an exam hall only if:
• You are registered
• You paid fees
• Exam time is valid

✅ Practical Use

• Reduces test cases
• Avoids missing conditions

✅ Important Exam-Writing Points

✓ Identify causes clearly
✓ Identify effects clearly
✓ Use AND/OR logic
✓ Convert graph → decision table

🔹 STEP-2: DIAGRAM / GRAPH (MANDATORY)

✓ Cause–Effect Graph (ASCII)

```
C1 ----\
        AND ----> E1 (Start Exam)
C2 ----/
        \
         AND
        /
C3 ----/

E2 = Exam Not Allowed
```

✓ Label Explanation

C1 = Student registered
C2 = Exam fee paid
C3 = Exam window open
E1 = Exam starts
E2 = Exam not allowed

✓ Decision Table

```
Conditions / Rules   R1  R2  R3  R4
----------------------------------
C1 (Registered)      T   T   F   T
C2 (Fee Paid)        T   F   T   T
C3 (Window Open)     T   T   T   F

Action
Start Exam           Y   N   N   N
```

🔹 STEP-3: MATHEMATICAL FORMULAS

Total combinations:

$2^3 = 8$

Reduced to **4 meaningful test cases**

🔹 STEP-4: NUMERICAL / CALCULATION

Only Rule-1 satisfies:

C1 = T
C2 = T
C3 = T

So exam can start.

🔹 STEP-5: FINAL ANSWER

👉 **FINAL ANSWER:**
4 test cases are sufficient using cause–effect graphing.

🔹 STEP-6: EXAM TIP

• Always draw AND gate
• Write both success and failure
• Don't forget decision table
• Many students forget "exam window" condition

🔷 Q27

🔹 QUESTION

(Chapter-06 & 07: Data Flow Testing)

For the following code, identify all **data-flow anomalies**.

```
1. x = 10
2. y = x + 5
3. x = 20
4. print(y)
```

🔹 STEP-1: THEORY EXPLANATION

✅ Intuition

We track **how variables are defined, used, and killed**.

✅ Basic Explanation

Symbols used:

- d = define
- u = use
- k = kill

✅ Exam Definition

Data-flow anomaly occurs when variable usage violates normal definition–use order.

✅ Real-Life Analogy

Writing money value, changing it, but spending old value.

✅ Practical Use

Detects logical bugs.

✅ Important Exam Points

✓ Track each variable separately
✓ Write d-u-k sequence

🔹 STEP-2: DATA FLOW DIAGRAM

```
x: d → u → d
y: d → u
```

✓ Label Explanation

x defined at line-1
x used at line-2
x redefined at line-3

y defined at line-2
y used at line-4

🔹 STEP-3: FORMULAS / SYMBOLS

Anomaly types:

• d-d (redefinition without use) ❌
• u-k (use after kill) ❌

🔹 STEP-4: CALCULATION

Check x:

x = 10 (d)
x used in y = x + 5 (u)
x redefined to 20 (d)

✓ No anomaly (x was used before redefinition)

Check y:

y defined → used → OK

🔹 STEP-5: FINAL ANSWER

👉 **FINAL ANSWER:**
No data-flow anomaly present.

🔹 STEP-6: EXAM TIP

• Write sequence explicitly
• Don't guess anomaly
• Students often falsely mark d-d here

🔷 Q28

🔹 QUESTION

(Chapter-04: MC/DC Coverage)

For the condition:

```
(A AND B AND C)
```

Design **minimum MC/DC test cases**.

🔹 STEP-1: THEORY EXPLANATION

✅ Intuition

Each condition must **independently affect output**.

✅ Basic Explanation

Change one condition → output must change
Others stay same

✅ Exam Definition

MC/DC requires each condition to independently influence decision outcome.

✅ Real-Life Analogy

Each switch should independently control a light.

✅ Practical Use

Used in safety-critical systems.

✅ Important Exam Points

✓ Independence proof required
✓ Minimum tests = N + 1

🔹 STEP-2: TRUTH TABLE (MANDATORY)

```
A B C | Output
----------------
T T T |    T
F T T |    F
T F T |    F
T T F |    F
```

🔹 STEP-3: FORMULAS WITH SYMBOLS

N = number of conditions = 3

MC/DC tests = N + 1 = 4

🔹 STEP-4: CALCULATION

• Change A (T→F) → output changes
• Change B (T→F) → output changes
• Change C (T→F) → output changes

Independence proven.

🔹 STEP-5: FINAL ANSWER

👉 **FINAL ANSWER:**
Minimum **4 MC/DC test cases** required.

🔹 STEP-6: EXAM TIP

• Always start with all TRUE
• Change one variable at a time
• Students wrongly write 8 tests

🔷 Q29

🔹 QUESTION

(Chapter-10: Debugging – Program Slicing)

Explain **Backward Program Slicing** with example.

🔹 STEP-1: THEORY EXPLANATION

✅ Intuition

We look **backward** to find what caused an error.

✅ Basic Explanation

Backward slice finds statements affecting a variable at a point.

✅ Exam Definition

Backward slicing identifies all statements that influence the value of a variable at a given program point.

✅ Real-Life Analogy

Tracing why your bank balance is low.

✅ Practical Use

Helps debugging faster.

✅ Important Exam Points

✓ Mention slicing criterion
✓ Mention variable and point

🔹 STEP-2: DIAGRAM (ASCII)

Program:

```
1. a = 5
2. b = a + 2
3. c = b + 3
4. print(c)
```

Backward Slice for `<print(c), c>`:

```
a → b → c → print
```

✓ Label Explanation

Only statements affecting `c` are included.

🔹 STEP-3: FORMULAS / SYMBOLS

Slicing Criterion = <statement, variable>

🔹 STEP-4: CALCULATION

c depends on b
b depends on a

So slice includes lines 1,2,3,4

🔹 STEP-5: FINAL ANSWER

👉 **FINAL ANSWER:**
Backward slice includes all statements influencing variable `c`.

◆ STEP-6: EXAM TIP

• Clearly write slicing criterion
• Draw dependency arrows
• Students confuse forward/backward slicing

◆ Q30

◆ QUESTION

(Chapter-14: System Testing – Performance Testing)

Explain **Stress Testing** with diagram and example.

◆ STEP-1: THEORY EXPLANATION

✅ Intuition

Test system beyond limits.

✅ Basic Explanation

We give **more load than expected**.

✅ Exam Definition

Stress testing evaluates system behavior under extreme load conditions.

✅ Real-Life Analogy

Overloading a bridge to see when it breaks.

✅ Practical Use

Ensures system reliability.

✅ Important Exam Points

✓ Mention overload
✓ Mention recovery

◆ STEP-2: DIAGRAM

```
Normal Load → Heavy Load → Extreme Load
     |            |             |
   Pass         Slow         Failure
```

✓ Label Explanation

Extreme load = stress condition

🔹 STEP-3: FORMULAS

No mathematical formula.

🔹 STEP-4: NUMERICAL EXAMPLE

If system supports 100 users:

Stress test with:
• 150 users
• 200 users

Check crash or recovery.

🔹 STEP-5: FINAL ANSWER

👉 **FINAL ANSWER:**
Stress testing checks system behavior under extreme load.

🔹 STEP-6: EXAM TIP

• Don’t confuse with load testing
• Draw simple flow
• Mention recovery behavior

Good — now I'll mark the **MOST-IMPORTANT QUESTIONS** from your 30.

These are selected based on:

✓ Previous year repetition
✓ University exam trends
✓ Weightage in papers
✓ Core concepts professors love
✓ High chance for long questions (10–15 marks)

I'll mark them as:

⭐ = Important
⭐⭐ = Very Important
⭐⭐⭐ = MOST IMPORTANT (must study first)

✅ 🔥 TOP PRIORITY (STUDY FIRST)

These are your **highest probability exam questions**.

⭐⭐⭐ Q1 — Cause–Effect Graph + Decision Table (Login / Exam eligibility)

Why important:

- Very frequently asked
- Tests logic + diagram
- Often 10–15 marks
- Professors love this topic

👉 **MUST PRACTICE drawing graph + table**

⭐⭐⭐ Q2 / Q25 — CFG + Cyclomatic Complexity + Independent Paths

Why important:

- Appears almost every year
- Easy scoring if practiced
- Direct numerical question

👉 Know:

- CFG drawing
- V(G)=D+1
- Independent paths

⭐ ⭐ ⭐ Q4 / Q28 — MC/DC Testing

Why important:

- High-weight topic
- Repeated in PYQs
- Often asked in numericals

👉 Remember:
MC/DC = N+1 tests

⭐ ⭐ ⭐ Q5 / Q22 / Q13 — Mutation Testing (Score Calculation)

Why important:

- Simple formula question
- Almost guaranteed in exams
- Easy marks

👉 Formula:
MS = (Killed / (Total − Equivalent)) ×100

⭐ ⭐ ⭐ Q7 / Q24 — Orthogonal Array Testing (OATS)

Why important:

- Favorite theory + table question
- Shows test reduction logic
- Frequently asked

👉 Focus:
Pairwise coverage concept

✅ ⭐ ⭐ VERY IMPORTANT

Study after top priority.

⭐⭐ Q6 / Q20 — Concolic Testing

Why important:

- Modern topic
- Often asked as theory + example
- Good for 10 marks

⭐⭐ Q8 / Q27 — Data Flow Testing (DU Chains / Anomalies)

Why important:

- Professors ask to test understanding
- Appears in mid-sems often

⭐⭐ Q9 / Q15 — Top-Down & Bottom-Up Integration

Why important:

- Classic long answer
- Easy to write with diagram

⭐⭐ Q21 — Integration Effort Formula

Why important:

- Direct numerical
- Rare but scoring

⭐⭐ Q23 / Q30 — Smoke & Stress Testing

Why important:

- Common theory questions
- Easy 5–10 marks

✅⭐ IMPORTANT (REVISION LEVEL)

Study if time remains.

⭐ Q3 / Q12 — Decision Tables (basic)

Good but simpler.

⭐ Q11 — FSM Testing

Sometimes asked, not every year.

⭐ Q14 — Alpha vs Beta Testing

Short theory question.

⭐ Q16 / Q17 — Call Graph & MM Path

Asked less frequently but useful.

⭐ Q18 — Data Flow Sequence

Low probability direct question.

✅ 🎯 SMART STUDY ORDER

If exam is near, follow this:

🥇 FIRST (Guaranteed topics)

1. CFG & Cyclomatic Complexity
2. Cause–Effect Graph
3. MC/DC
4. Mutation Testing
5. OATS

🥈 SECOND

6. Concolic Testing
7. Data Flow Testing
8. Integration Testing (Top/Bottom)

🥉 THIRD

9. System Testing types
10. FSM
11. Debugging & Slicing

✅ REALISTIC EXAM PREDICTION

From experience, your paper will likely include:

✓ 1 CFG question
✓ 1 Cause–Effect graph
✓ 1 Mutation score
✓ 1 MC/DC
✓ 1 OATS or Decision Table
✓ 1 System testing theory

🚀 SOFTWARE TESTING – 1-PAGE SUPER CHEAT SHEET

🔥 MOST ASKED FORMULAS

✅ Cyclomatic Complexity

```
V(G) = Decisions + 1
V(G) = E − N + 2
```

👉 Minimum test cases = V(G)

✅ MC/DC

```
Minimum Tests = N + 1
```

N = number of conditions
👉 Each condition must independently affect output

✅ Mutation Score

```
MS = (Killed / (Total − Equivalent)) × 100
```

👉 Always subtract equivalent mutants

✅ OATS (Orthogonal Array)

```
Tests without OATS = L^F
```

L = levels
F = factors
👉 OATS = pairwise coverage → fewer tests

✅ **Integration Effort**

```
Sessions = Nodes – Leaves + Edges
```

🔥 CFG (Control Flow Graph)

👉 Each **if/loop = 1 decision**
👉 Independent paths = V(G)

Exam Steps:

1. Draw CFG
2. Count decisions
3. Apply formula

🔥 Cause–Effect Graph

Steps:

```
Causes → Graph → Decision Table → Test Cases
```

Logic:
AND = all required
OR = any required
NOT = opposite

👉 Always convert to decision table

🔥 Data Flow Testing

Symbols:

```
d = define
u = use
k = kill
```

Anomalies:

```
d-d ❌
u-k ❌
d-u ✓
```

DU Chain:
Definition → Use (no redefinition)

🔥 Integration Testing

Top-Down

• Start from main module
• Use STUBS

Bottom-Up

• Start from leaf
• Use DRIVERS

Sandwich

• Mix of both

🔥 Concolic Testing

```
Concrete + Symbolic
```

Steps:

1. Run with input
2. Track constraints
3. Solve constraints
4. Generate new tests

👉 Used for path coverage

🔥 System Testing

Alpha

Inside company

Beta

Real users

Acceptance

Customer approves

🔥 Smoke Testing

Quick basic test before deep testing

```
If smoke fails → STOP testing
```

Check:
• Login
• Buttons
• DB connection

🔥 Stress Testing

Test beyond limits

Example:
Capacity = 100 users
Test with 200 users

Goal:
Crash & recovery behavior

🔥 Debugging – Slicing

Backward slicing:

```
Trace backward from error
```

Find statements affecting value

🧠 ULTRA MEMORY TRICKS

Cyclomatic → Decisions+1
MC/DC → N+1
Mutation → Killed/(Total−Eq)
OATS → Pairwise
Cause–Effect → Graph→Table→Tests

🎯 30-MINUTE REVISION PLAN

If time very less:

Revise ONLY:
✓ CFG
✓ Cause–Effect
✓ MC/DC
✓ Mutation
✓ OATS

👉 Covers ~70–80% paper

🏆 GOLDEN EXAM TIPS

✓ Draw diagrams
✓ Write formulas clearly
✓ Show steps in numericals
✓ Use keywords
✓ Attempt easy numericals first
✓ Don't leave MC/DC question